

Lab 4: Parallel task decomposition implementation and analysis

Parallelism

Universitat Politècnica de Catalunya (UPC)

Facultat d'Informàtica de Barcelona (FIB)

Authors:

FERRAN MESAS

`ferran.mesas@estudiantat.upc.edu`

ISHAK FELFOUL

`ishak.felfoul@estudiantat.upc.edu`

May 22, 2026

Table of Contents

1	Experimental Setup	4
1.1	Correctness validation	4
1.2	Performance analysis methodology	4
2	Iterative Task Decomposition	5
2.1	Original Tile Iterative Task Decomposition	6
2.1.1	Task decomposition	6
2.1.2	Code	6
2.1.3	Modelfactors analysis	8
2.1.4	Paraver analysis	10
2.1.5	Strong scalability	12
2.2	Fine-Grain Iterative Task Decomposition	14
2.2.1	Task decomposition	14
2.2.2	Code	14
2.2.3	Modelfactors analysis	18
2.2.4	Paraver analysis	20
2.2.5	Strong scalability	22
2.3	Comparison between Iterative Strategies	24
3	Recursive Task Decomposition	25
3.1	Leaf Recursive Strategy	26
3.1.1	Task Decomposition	26

3.1.2	Source Code Implementation	26
3.1.3	Modelfactors Analysis	29
3.1.4	Paraver Analysis	30
3.1.5	Strong Scaling	32
3.2	Improved Leaf Recursive Strategy	34
3.2.1	Task Decomposition and Cutoff Strategy	34
3.2.2	Source Code Implementation	34
3.2.3	Modelfactors Analysis	37
3.2.4	Paraver Analysis	38
3.2.5	Strong Scaling	40
3.2.6	Comparison with the Basic Leaf Version	41
3.3	Recursive Tree Strategy	42
3.3.1	Task Decomposition	42
3.3.2	Source Code Implementation	42
3.3.3	Modelfactors Analysis	45
3.3.4	Paraver Analysis	46
3.3.5	Strong Scaling	48
3.3.6	Comparison with Improved Leaf	49
3.4	Comparison Between Recursive Strategies	49
4	Final Strategy Comparison	51
4.1	Comparison Table	51
4.2	Comparison between all Parallel Strategies	51

4.3	Conclusions	52
-----	-----------------------	----

1. Experimental Setup

This laboratory studies several OpenMP task decompositions of the Mandelbrot computation. The implementations are based on the iterative and recursive strategies previously analysed with Tareador in Lab 3.

1.1. Correctness validation

Before analysing performance, every parallel implementation was checked against the corresponding sequential version. For the iterative strategies, the reference output was generated with:

```
sbatch submit-seq.sh mandel-seq-iter -h -o -i 10000
```

For the recursive strategies, the reference output was generated with:

```
sbatch submit-seq.sh mandel-seq-rec -h -o -i 10000
```

Each parallel version was then executed with the same parameters, and the generated `mandel_image.jpg` and `mandel_histogram.out` files were compared with the sequential outputs using `cmp`. All results presented in this report correspond to implementations that passed this correctness check.

1.2. Performance analysis methodology

For each strategy, we include the source file name, the Model factors tables, the Paraver analysis of the execution trace with 16 threads, and the strong scalability results. The Paraver views used are *Useful/Instantaneous parallelism* and *OpenMP tasking/explicit tasks function created and executed*.

2. Iterative Task Decomposition

This section analyses the two iterative OpenMP implementations of the Mandelbrot computation: the original tile decomposition and the fine-grain decomposition.

2.1. Original Tile Iterative Task Decomposition

2.1.1. Task decomposition

The original tile iterative strategy creates one explicit OpenMP task for each image tile. Each task first computes the border points of its tile and checks whether all of them have the same dwell value. If the tile is uniform, the whole tile is filled with that value. Otherwise, the complete tile is computed pixel by pixel.

This implementation corresponds to the original tile strategy analysed in Lab 3, but now implemented using real OpenMP tasks.

2.1.2. Code

The implementation is provided in:

`mandel-omp-iter-original-tile.c`

The first change is to enclose the tile loop inside a parallel region and a single construct. This way, all OpenMP threads are created, but only one of them generates the tile tasks.

Listing 1: Task generation structure for the original tile version.

```
#pragma omp parallel
#pragma omp single
for (int y = 0; y < ROWS; y += TILE)
    for (int x = 0; x < COLS; x += TILE) {
#pragma omp task firstprivate(x, y)
    {
        int equal = 1;

        /* Border checks and tile computation/filling */
    }
}
```

The `firstprivate(x,y)` clause is needed because tasks may execute after the loop variables have changed. Each task must keep the coordinates of the tile it is responsible for. The variable `equal` is declared inside the task so that each tile has its own private copy.

The histogram is shared by all tasks. Therefore, histogram updates must be protected with `atomic` to avoid data races when two tasks update the same bin.

Listing 2: Atomic histogram update.

```
if (output2histogram) {  
#pragma omp atomic  
    histogram[M[py]][px] - 1]++;  
}
```

For the uniform-tile case, the update accumulates the whole tile contribution:

```
if (output2histogram) {  
#pragma omp atomic  
    histogram[M[y]][x] - 1] += (TILE * TILE);  
}
```

The display operations also access shared X11 state. To preserve correctness when the `-d` option is used, these operations are protected with a critical section.

Listing 3: Critical section for display operations.

```
if (output2display) {  
    if (setup_return == EXIT_SUCCESS) {  
#pragma omp critical(display_output)  
        {  
            XSetForeground(display, gc, color);  
            XDrawPoint(display, win, gc, px, py);  
        }  
    }  
}
```

No explicit `taskwait` is required in this version, since no code inside a tile task depends on other tile tasks. The implicit synchronization at the end of the parallel region ensures that all tile tasks have completed before returning from `mandel_tiled`.

2.1.3. Modelfactors analysis

Overview of whole program execution metrics										
Number of threads	1	2	4	8	12	16	20	24	28	32
Elapsed time (sec)	3.47	1.95	1.02	0.73	0.72	0.72	0.72	0.72	0.72	0.72
Speedup	1.00	1.78	3.39	4.73	4.81	4.81	4.80	4.81	4.80	4.80
Efficiency	1.00	0.89	0.85	0.59	0.40	0.30	0.24	0.20	0.17	0.15

Table 1: Analysis done on Fri May 22 06:04:15 AM CEST 2026, par2314

Figure 1: Modelfactors overview and efficiency metrics for the original tile iterative version.

Overview of the Efficiency metrics in parallel fraction, $\phi=99.99\%$										
Number of threads	1	2	4	8	12	16	20	24	28	32
Global efficiency	100.00%	88.86%	84.86%	59.17%	40.10%	30.09%	24.03%	20.05%	17.15%	15.01%
Parallelization strategy efficiency	100.00%	88.89%	84.92%	59.35%	40.32%	30.34%	24.36%	20.33%	17.43%	15.40%
Load balancing	100.00%	88.90%	84.98%	59.42%	40.35%	30.37%	24.41%	20.37%	17.48%	15.44%
In execution efficiency	100.00%	99.99%	99.93%	99.88%	99.92%	99.89%	99.81%	99.80%	99.70%	99.73%
Scalability for computation tasks	100.00%	99.97%	99.93%	99.70%	99.46%	99.17%	98.64%	98.64%	98.41%	97.49%
IPC scalability	100.00%	100.01%	100.00%	99.99%	99.98%	99.98%	99.98%	99.97%	99.98%	99.97%
Instruction scalability	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
Frequency scalability	100.00%	99.95%	99.92%	99.71%	99.47%	99.20%	98.67%	98.66%	98.44%	97.52%

Table 2: Analysis done on Fri May 22 06:04:15 AM CEST 2026, par2314

Statistics about explicit tasks in parallel fraction										
Number of threads	1	2	4	8	12	16	20	24	28	32
Number of explicit tasks executed (total)	64.0	64.0	64.0	64.0	64.0	64.0	64.0	64.0	64.0	64.0
LB (number of explicit tasks executed)	1.0	0.94	0.64	0.44	0.31	0.24	0.19	0.16	0.13	0.12
LB (time executing explicit tasks)	1.0	0.89	0.85	0.59	0.4	0.3	0.24	0.2	0.17	0.15
Time per explicit task (average us)	54153.59	54171.22	54178.29	54292.18	54386.1	54484.81	54713.71	54592.8	54659.91	55033.5
Overhead per explicit task (synch %)	0.0	12.49	17.68	68.35	148.01	229.76	310.82	393.22	475.33	552.76
Overhead per explicit task (sched %)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.01	0.01
Number of taskwait/taskgroup (total)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

Table 3: Analysis done on Fri May 22 06:04:15 AM CEST 2026, par2314

Figure 2: Modelfactors explicit task statistics for the original tile iterative version.

The Modelfactors results show that the parallel fraction is very high, approximately 99.99%, so the scalability problem does not come from a large sequential part. Instead, the main limitation is load imbalance.

The version creates only 64 explicit tasks, one per tile. Since the Mandelbrot computation has highly irregular cost, some tiles are much more expensive than others. This is reflected in the load balancing metric, which decreases from 84.98% with 4 threads to only 15.44% with 32 threads. In contrast, the in-execution efficiency remains close to 100%, which indicates that the main problem is not task scheduling overhead, but the uneven distribution of work among tasks.

The explicit-task statistics confirm the same issue. The number of tasks is fixed at 64 for all executions, and the load balance in task execution time follows the same degradation observed in the global efficiency metrics.

2.1.4. Paraver analysis

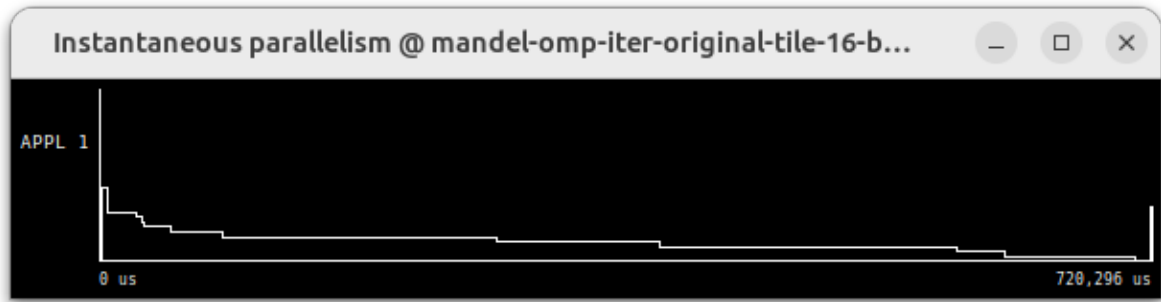


Figure 3: Useful/Instantaneous parallelism for the original tile iterative version with 16 threads.

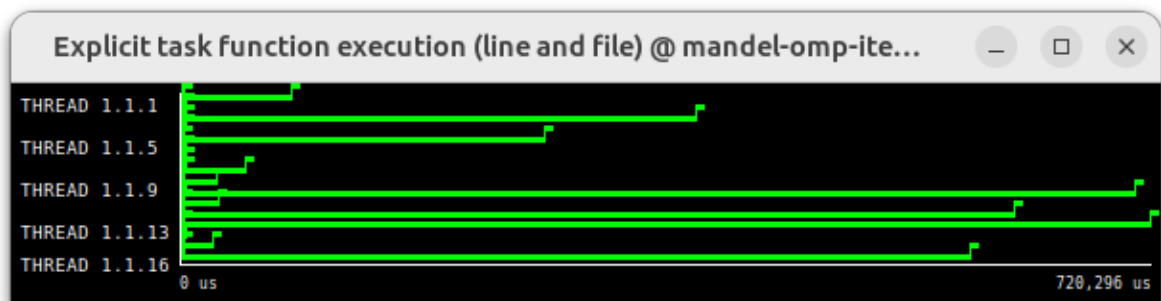


Figure 4: Explicit task execution for the original tile iterative version with 16 threads.

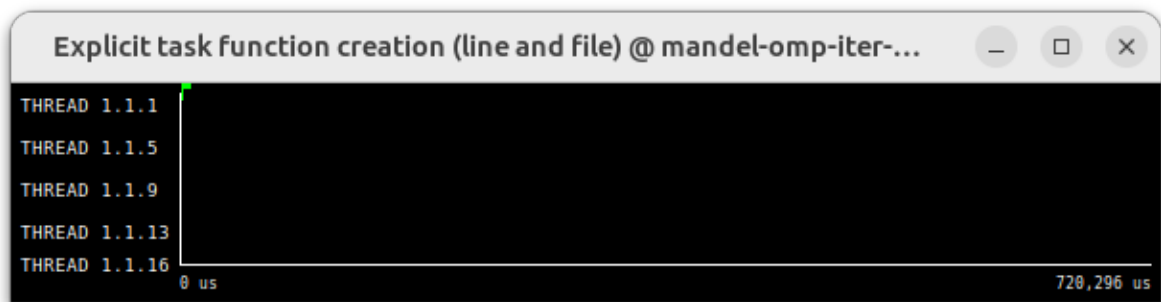


Figure 5: Explicit task creation for the original tile iterative version with 16 threads.

The Paraver trace is consistent with the Modelfactors results. The instantaneous parallelism is not sustained throughout the execution: once the cheaper tile tasks finish, some threads stop having useful work while other threads continue executing the most expensive tiles.

The explicit task execution view shows a small number of relatively coarse tasks. This matches the 64 explicit tasks reported by Modelfactors. The task creation view also

shows that task generation is concentrated at the beginning of the computation, after which the runtime only has those tile tasks to distribute. Since the number of tasks is small and their cost is very irregular, the runtime cannot balance the work effectively at high thread counts.

2.1.5. Strong scalability

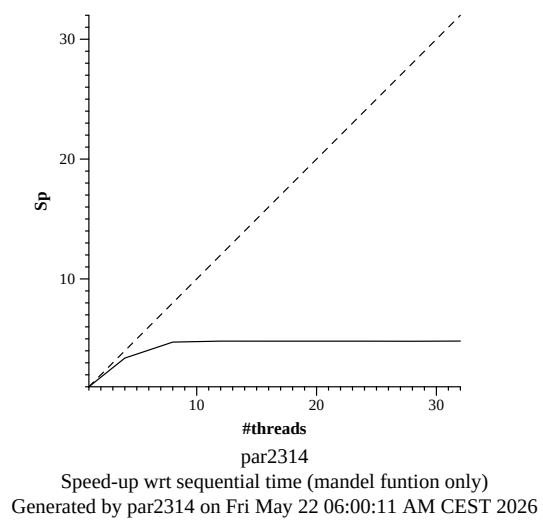


Figure 6: Strong scalability of the original tile iterative version.

Threads	1	4	8	12	16	32
Elapsed time (s)	3.462507	1.019928	0.732130	0.719861	0.720354	0.719224
Speedup	0.9997	3.3939	4.7280	4.8086	4.8053	4.8129

Table 1: Strong scalability results for the original tile iterative version.

The strong scalability results show that the original tile version improves up to 8–12 threads, but then saturates. The execution time decreases from 3.46 seconds with one thread to about 0.72 seconds with 12 threads, but adding more threads does not improve the execution time. The maximum speedup obtained is around 4.8.

This behaviour is explained by the limited number of coarse tasks and their irregular cost. With more threads, there are not enough balanced tasks to keep all threads busy, so the load imbalance dominates the execution.

2.2. Fine-Grain Iterative Task Decomposition

2.2.1. Task decomposition

The fine-grain iterative strategy starts from the original tile decomposition, but introduces additional tasks inside each tile. Therefore, the outer task per tile is preserved, as required by the statement, but the work inside each tile is split into smaller tasks.

For each tile, two tasks are created to compute and check the vertical and horizontal borders. After these two checks finish, the program decides whether the tile is uniform. If it is uniform, the tile is filled using row-level tasks. Otherwise, the tile is computed using one task per row.

2.2.2. Code

The implementation is provided in:

`mandel-omp-iter-fine-grain.c`

The first part is similar to the original tile version: the tile loop is enclosed in a `parallel` region and a `single` construct, and one outer task is still created for each tile.

Listing 4: Outer tile task preserved in the fine-grain version.

```
#pragma omp parallel
#pragma omp single
for (int y = 0; y < ROWS; y += TILE)
    for (int x = 0; x < COLS; x += TILE) {
#pragma omp task firstprivate(x, y)
    {
        int ref = pixel_dwell(..., x, y, ...);
        int equal_horizontal = 1;
        int equal_vertical = 1;
        M[y][x] = ref;

        /* Fine-grain tasks inside the tile */
    }
}
```

The variable `ref` stores the value of the reference corner of the tile. This avoids using `M[y][x]` as a shared value later inside the row tasks. The variables `equal_horizontal`

and `equal_vertical` separate the result of the two border checks, avoiding a race on a single shared `equal` variable.

The border checks are split into two independent tasks:

Listing 5: Border-check tasks in the fine-grain version.

```
#pragma omp task shared(equal_vertical)
for (int py = y + 1; py < y + TILE - 1; py++) {
    M[py][x] = pixel_dwell(..., x, py, ...);
    M[py][x + TILE - 1] = pixel_dwell(..., x + TILE - 1, py, ...);

    equal_vertical = equal_vertical & (ref == M[py][x]);
    equal_vertical = equal_vertical & (ref == M[py][x + TILE - 1]);
}

#pragma omp task shared(equal_horizontal)
for (int px = x; px < x + TILE; px++) {
    M[y][px] = pixel_dwell(..., px, y, ...);
    M[y + TILE - 1][px] = pixel_dwell(..., px, y + TILE - 1, ...);

    equal_horizontal = equal_horizontal & (ref == M[y][px]);
    equal_horizontal = equal_horizontal & (ref == M[y + TILE - 1][px]);
}

#pragma omp taskwait
```

The `taskwait` is necessary because the parent tile task must wait for both border-check tasks before reading `equal_vertical` and `equal_horizontal`. Without this synchronization, the decision about whether the tile is uniform could be made before the checks finish.

In the uniform case, one task is created per row to fill the tile:

Listing 6: Row-level fill tasks for uniform tiles.

```
if (equal_vertical & equal_horizontal) {
    long color = (long)((ref - 1) * scale_color) + min_color;

    if (output2histogram) {
#pragma omp atomic
        histogram[ref - 1] += (TILE * TILE);
    }

    for (int py = y; py < y + TILE; py++)
```



```

#pragma omp task firstprivate(py, x, ref, color)
    for (int px = x; px < x + TILE; px++) {
        M[py][px] = ref;

        if (output2display) {
            if (setup_return == EXIT_SUCCESS)
#pragma omp critical(display_output)
            {
                XSetForeground(display, gc, color);
                XDrawPoint(display, win, gc, px, py);
            }
        }
    }
}

```

In the non-uniform case, one task is created per row to compute the tile:

Listing 7: Row-level computation tasks for non-uniform tiles.

```

else {
    for (int py = y; py < y + TILE; py++)
#pragma omp task firstprivate(py, x)
        for (int px = x; px < x + TILE; px++) {
            M[py][px] = pixel_dwell(..., px, py, ...);

            if (output2histogram) {
#pragma omp atomic
                histogram[M[py][px] - 1]++;
            }

            if (output2display)
#pragma omp critical(display_output)
            {
                long color = (long)((M[py][px] - 1) * scale_color) + min_c
                if (setup_return == EXIT_SUCCESS) {
                    XSetForeground(display, gc, color);
                    XDrawPoint(display, win, gc, px, py);
                }
            }
        }
    }
}

```

As in the original tile version, histogram updates are protected with `atomic`, and X11

display operations are protected with a critical section. No final `taskwait` is needed after the row tasks, because the enclosing parallel region waits for all pending tasks before `mandel_tiled` returns.

2.2.3. Modelfactors analysis

Overview of whole program execution metrics										
Number of threads	1	2	4	8	12	16	20	24	28	32
Elapsed time (sec)	3.48	1.75	0.88	0.45	0.31	0.23	0.19	0.16	0.15	0.13
Speedup	1.00	1.99	3.95	7.77	11.42	14.82	18.05	21.24	23.94	26.60
Efficiency	1.00	0.99	0.99	0.97	0.95	0.93	0.90	0.88	0.86	0.83

Table 1: Analysis done on Fri May 22 06:20:07 AM CEST 2026, par2314

Figure 7: Modelfactors overview and efficiency metrics for the fine-grain iterative version.

Overview of the Efficiency metrics in parallel fraction, $\phi=99.99\%$										
Number of threads	1	2	4	8	12	16	20	24	28	32
Global efficiency	99.94%	99.29%	98.75%	97.09%	95.23%	92.75%	90.35%	88.59%	85.67%	83.30%
Parallelization strategy efficiency	99.94%	99.42%	98.90%	97.46%	95.67%	93.55%	91.32%	89.60%	86.78%	85.07%
Load balancing	100.00%	99.99%	99.70%	98.96%	98.03%	97.50%	94.27%	93.41%	89.91%	89.31%
In execution efficiency	99.94%	99.44%	99.19%	98.49%	97.59%	95.95%	96.87%	95.92%	96.52%	95.25%
Scalability for computation tasks	100.00%	99.87%	99.85%	99.63%	99.54%	99.14%	98.93%	98.88%	98.72%	97.91%
IPC scalability	100.00%	99.87%	99.86%	99.83%	99.83%	99.80%	99.79%	99.78%	99.79%	99.80%
Instruction scalability	100.00%	100.10%	100.10%	100.10%	100.09%	100.09%	100.09%	100.09%	100.09%	100.09%
Frequency scalability	100.00%	99.90%	99.90%	99.70%	99.62%	99.25%	99.05%	99.00%	98.84%	98.02%

Table 2: Analysis done on Fri May 22 06:20:07 AM CEST 2026, par2314

Statistics about explicit tasks in parallel fraction										
Number of threads	1	2	4	8	12	16	20	24	28	32
Number of explicit tasks executed (total)	8384.0	8384.0	8384.0	8384.0	8384.0	8384.0	8384.0	8384.0	8384.0	8384.0
LB (number of explicit tasks executed)	1.0	0.95	0.64	0.46	0.32	0.37	0.34	0.46	0.41	0.25
LB (time executing explicit tasks)	1.0	1.0	1.0	0.99	0.98	0.97	0.94	0.94	0.9	0.89
Time per explicit task (average us)	414.62	415.25	415.33	416.12	416.29	417.51	418.05	417.6	417.8	420.57
Overhead per explicit task (synch %)	0.0	0.37	0.69	1.65	2.63	3.82	5.33	6.37	7.94	8.86
Overhead per explicit task (sched %)	0.05	0.19	0.37	0.86	1.62	2.58	3.5	4.46	6.21	7.03
Number of taskwait/taskgroup (total)	64.0	64.0	64.0	64.0	64.0	64.0	64.0	64.0	64.0	64.0

Table 3: Analysis done on Fri May 22 06:20:07 AM CEST 2026, par2314

Figure 8: Modelfactors explicit task statistics for the fine-grain iterative version.

The Modelfactors results show a clear improvement with respect to the original tile version. The number of explicit tasks increases from 64 to 8384, which gives the OpenMP runtime many more scheduling opportunities.

This finer decomposition significantly improves the load balance. While the original tile version was limited by a few very expensive tile tasks, the fine-grain version splits those tiles into smaller units of work. As a result, the global efficiency remains high even for 32 threads, reaching approximately 83%. The remaining loss of efficiency is mainly due to the overhead of creating and scheduling many more tasks, but this overhead is compensated by the much better load balance.

2.2.4. Paraver analysis



Figure 9: Useful/Instantaneous parallelism for the fine-grain iterative version with 16 threads.

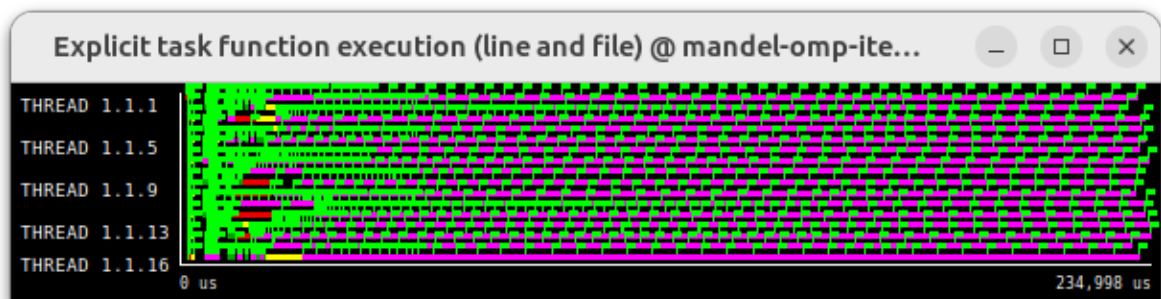


Figure 10: Explicit task execution for the fine-grain iterative version with 16 threads.

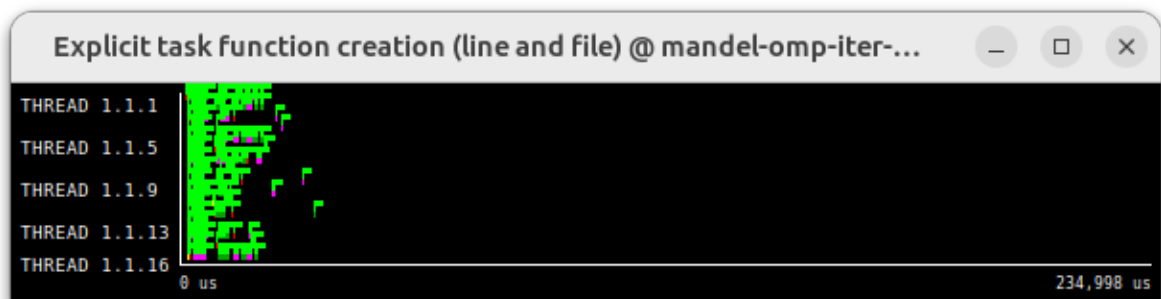


Figure 11: Explicit task creation for the fine-grain iterative version with 16 threads.

The Paraver trace confirms the improvement observed in Modelfactors. The instantaneous parallelism remains much closer to the available number of threads during most of the execution. This indicates that the runtime has enough ready tasks to keep the threads busy.

The explicit task execution view is much denser than in the original tile version, showing that the work has been divided into many smaller tasks. The task creation view also

shows a larger number of task creation events, corresponding to the border-check tasks and the row-level fill or computation tasks. This explains why the fine-grain version achieves better scalability: the runtime can distribute the work more evenly among the threads.

2.2.5. Strong scalability

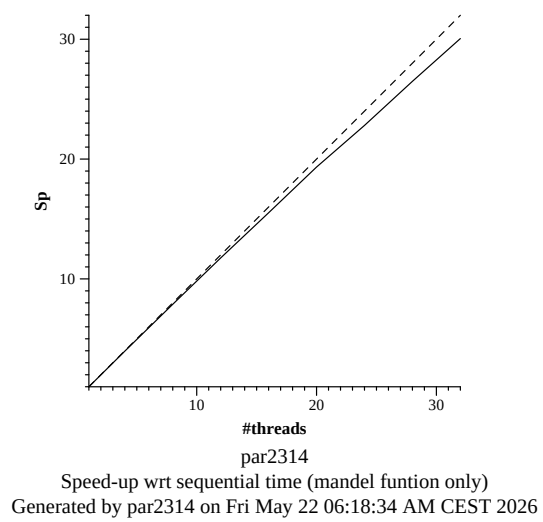


Figure 12: Strong scalability of the fine-grain iterative version.

Threads	1	4	8	12	16	32
Elapsed time (s)	3.473498	0.871647	0.439175	0.294578	0.223028	0.115157
Speedup	0.9964	3.9708	7.8810	11.7495	15.5189	30.0559

Table 2: Strong scalability results for the fine-grain iterative version.

The fine-grain version scales much better than the original tile version. The execution time decreases from 3.47 seconds with one thread to 0.115 seconds with 32 threads. This corresponds to a speedup of approximately 30.

Unlike the original tile version, this implementation does not saturate at 8–12 threads. The additional fine-grain tasks reduce the load imbalance and allow the runtime to exploit the available threads more effectively. Therefore, among the iterative strategies, the fine-grain version is clearly the best implementation.

2.3. Comparison between Iterative Strategies

The original tile version creates only one explicit task per tile, resulting in 64 tasks. This decomposition has low task creation overhead, but it suffers from strong load imbalance because the cost of each tile depends on the region of the Mandelbrot set. Some tiles are very cheap, while others require many calls to `pixel_dwell()` with a high number of iterations.

The fine-grain version keeps the outer tile task, but splits the work inside each tile into smaller tasks. In particular, it creates separate tasks for the border checks and row-level tasks for the fill or computation phase. This increases the number of explicit tasks to 8384, giving the OpenMP runtime more opportunities to distribute work among threads.

The difference is clearly visible in the strong scalability results. The original tile version saturates around 8–12 threads and reaches a maximum speedup of approximately 4.8. In contrast, the fine-grain version continues scaling up to 32 threads, reaching a speedup of approximately 30.

Version	1	4	8	12	16	32
Original tile	3.462507	1.019928	0.732130	0.719861	0.720354	0.719224
Fine grain	3.473498	0.871647	0.439175	0.294578	0.223028	0.115157

Table 3: Elapsed execution time comparison between the iterative versions.

Version	1	4	8	12	16	32
Original tile	0.9997	3.3939	4.7280	4.8086	4.8053	4.8129
Fine grain	0.9964	3.9708	7.8810	11.7495	15.5189	30.0559

Table 4: Speedup comparison between the iterative versions.

Overall, the fine-grain implementation is the best iterative strategy. Although it creates many more tasks, the improvement in load balance is much more important than the additional task management overhead.

3. Recursive Task Decomposition

The recursive implementations exploit the divide-and-conquer structure of the Mandelbrot program. Starting from the full image, the recursive function checks whether the border of the current region has a uniform dwell value. If the region is uniform, the full region can be filled with the same value. Otherwise, the region is divided recursively until the base case is reached.

We implemented three recursive strategies: the basic leaf strategy, an improved leaf strategy with a cutoff, and the recursive tree strategy. All three versions preserve the same output behaviour as the sequential recursive program, including image generation, histogram generation, and display support.

Correctness was verified by comparing the image and histogram generated by each OpenMP version against the sequential recursive reference:

Listing 8: Correctness checks for the recursive versions.

```
cmp mandel_image_seq_rec.jpg mandel_image_rec_leaf.jpg
cmp mandel_histogram_seq_rec.out mandel_histogram_rec_leaf.out

cmp mandel_image_seq_rec.jpg mandel_image_rec_improved_leaf.jpg
cmp mandel_histogram_seq_rec.out mandel_histogram_rec_improved_leaf.out

cmp mandel_image_seq_rec.jpg mandel_image_rec_tree.jpg
cmp mandel_histogram_seq_rec.out mandel_histogram_rec_tree.out
```

All comparisons produced no output, so the three recursive OpenMP versions are binary-equivalent to the sequential recursive reference.

3.1. Leaf Recursive Strategy

3.1.1. Task Decomposition

The leaf recursive strategy keeps the recursive traversal sequential. OpenMP tasks are only created at the leaves of the recursion, that is, when the algorithm either finds a uniform region that can be filled directly, or when it reaches the base case $\text{NCols} \leq \text{TILE}$ and the pixels must be computed individually.

This strategy delays parallelism until the deepest levels of the recursion. Therefore, although it creates several tasks, the task generation is not exposed early enough to keep many threads active.

3.1.2. Source Code Implementation

The recursive computation is launched from a single thread inside a parallel region:

Listing 9: Starting the recursive computation in the leaf version.

```
#pragma omp parallel
#pragma omp single
mandel_tiled_rec((int (*)[width])Hmatrix, height, width, 0, 0,
                real_min, imag_min, real_max, imag_max,
                scale_real, scale_imag, maxiter);
```

When a uniform region is detected, a task is created to fill the full region. The value to fill is stored in a local variable before creating the task. This avoids reading $M[y][x]$ inside a task after the parent recursive call has continued.

Listing 10: Task creation for the uniform fill case.

```
if (equal) {
    int value = M[y][x];

#pragma omp task firstprivate(M, NRows, NCols, y, x, value)
    {
        long color = (long)((value - 1) * scale_color) + min_color;

        if (output2histogram) {
#pragma omp atomic
            histogram[value - 1] += (NRows * NCols);
        }
    }
}
```

```

    for (int py = y; py < y + NRows; py++)
        for (int px = x; px < x + NCols; px++) {
            M[py][px] = value;

            if (output2display)
#pragma omp critical(display_output)
            {
                if (setup_return == EXIT_SUCCESS) {
                    XSetForeground(display, gc, color);
                    XDrawPoint(display, win, gc, px, py);
                }
            }
        }
    }
}

```

When the base case is reached, a task computes the pixels of the leaf region:

Listing 11: Task creation for the base-case computation.

```

if (NCols <= TILE) {
#pragma omp task firstprivate(M, NRows, NCols, y, x, CminR, CminI, CmaxR, CmaxI)
    for (int py = y; py < y + NRows; py++)
        for (int px = x; px < x + NCols; px++) {
            M[py][px] = pixel_dwell(COLS, ROWS, CminR, CminI,
                                    CmaxR, CmaxI, px, py,
                                    scale_real, scale_imag, maxiter);

            if (output2histogram) {
#pragma omp atomic
                histogram[M[py][px] - 1]++;
            }

            if (output2display) {
                long color = (long)((M[py][px] - 1) * scale_color) + min_color;

                if (setup_return == EXIT_SUCCESS)
#pragma omp critical(display_output)
                {
                    XSetForeground(display, gc, color);
                    XDrawPoint(display, win, gc, px, py);
                }
            }
        }
    }
}

```

```

    }
}

```

The `firstprivate` clauses are necessary because tasks may execute after the parent recursive call has returned. The matrix pointer `M` is copied as a pointer value, so all tasks still write to the same image matrix. Histogram updates are protected with `atomic`, and display operations are protected with the named critical section `display_output`.

3.1.3. Modelfactors Analysis

Overview of whole program execution metrics										
Number of threads	1	2	4	8	12	16	20	24	28	32
Elapsed time (sec)	1.83	1.41	1.41	1.41	1.41	1.41	1.41	1.41	1.41	1.41
Speedup	1.00	1.30	1.30	1.30	1.30	1.30	1.29	1.29	1.29	1.29
Efficiency	1.00	0.65	0.32	0.16	0.11	0.08	0.06	0.05	0.05	0.04

Table 1: Analysis done on Fri May 22 10:26:14 AM CEST 2026, par2314

Figure 13: Execution metrics for the leaf recursive version.

The leaf version creates 3013 explicit tasks, with an average task duration of approximately 142 microseconds. Despite this number of tasks, the speedup saturates around 1.30. The global efficiency drops to 8.09% with 16 threads and to 4.04% with 32 threads.

The main limitation is not correctness but task granularity and task exposure. The recursive traversal is still mostly sequential, and tasks are created only after reaching the deepest recursive levels. As a result, the available parallelism is insufficient and the runtime overhead dominates the small amount of useful work per task.

3.1.4. Paraver Analysis

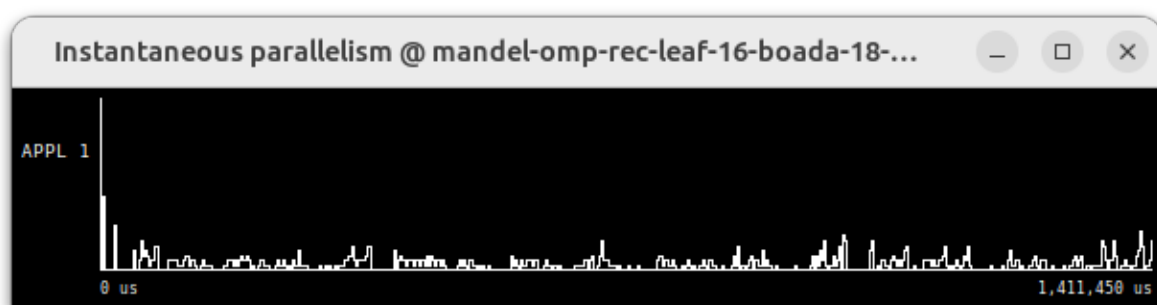


Figure 14: Instantaneous parallelism for the leaf recursive version with 16 threads.

The instantaneous parallelism view shows poor thread utilization. Parallelism appears as short and irregular peaks, but it is not sustained for most of the execution. This confirms that tasks are exposed too late and are not enough to keep 16 threads busy.

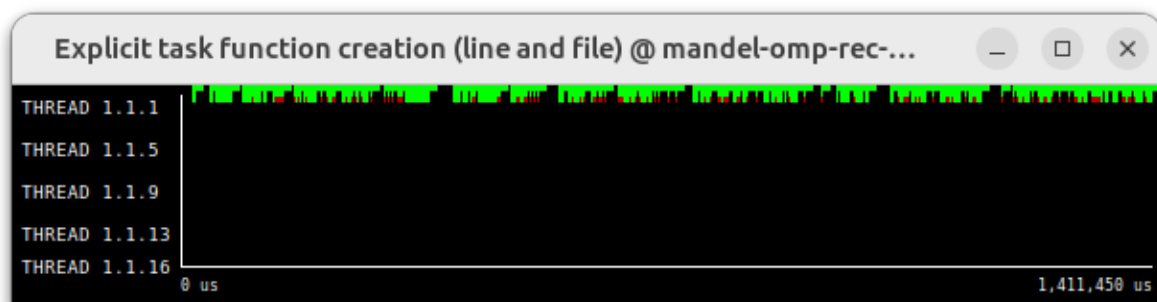


Figure 15: Task creation view for the leaf recursive version with 16 threads.



Figure 16: Task execution view for the leaf recursive version with 16 threads.

The task creation and execution views show many short tasks, but the work is not distributed in a way that produces sustained parallelism. This matches the ModelFactors result: the strategy has a high parallel fraction, but the load balance and effective efficiency are poor when the number of threads increases.

3.1.5. Strong Scaling

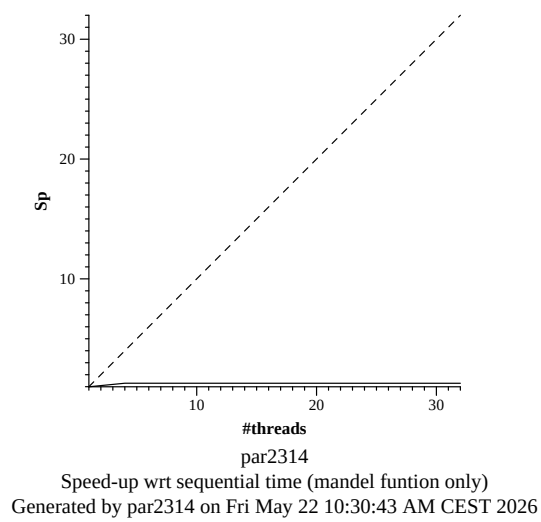


Figure 17: Strong scaling for the leaf recursive version.

Table 5: Strong scaling results for the leaf recursive version.

Threads	1	4	8	12	16	20	24	28	32
Time (s)	1.825030	1.405713	1.406957	1.407350	1.408503	1.408079	1.409933	1.408903	1.409785
Speedup	0.993	1.289	1.288	1.287	1.286	1.287	1.285	1.286	1.285

The speedup reaches approximately 1.29 and then remains flat. This confirms that the basic leaf version does not scale: once the limited available parallelism is consumed, adding more threads does not reduce the execution time.

3.2. Improved Leaf Recursive Strategy

3.2.1. Task Decomposition and Cutoff Strategy

The improved leaf strategy introduces a cutoff to control task granularity:

Listing 12: Cutoff used in the improved leaf version.

```
#define CUTOFF 2
```

The objective is to avoid creating thousands of very small final-leaf tasks. Instead, the recursion is followed sequentially until the cutoff depth is reached. At `depth == CUTOFF`, a task is created for a whole recursive subtree. Inside that task, the recursive computation continues normally.

This produces fewer but larger tasks. Therefore, the improved leaf version reduces task creation overhead compared with the basic leaf strategy, at the cost of exposing less total parallelism.

3.2.2. Source Code Implementation

The recursive function receives an additional depth parameter:

Listing 13: Recursive function with depth parameter.

```
void mandel_tiled_rec(int M[ROWS][COLS], int NRows, int NCols,
                    int start_fil, int start_col,
                    double CminR, double CminI,
                    double CmaxR, double CmaxI,
                    double scale_real, double scale_imag,
                    int maxiter, int depth)
```

The initial call starts with depth zero:

Listing 14: Initial call in the improved leaf version.

```
#pragma omp parallel
#pragma omp single
mandel_tiled_rec((int (*)[width])Hmatrix, height, width, 0, 0,
                real_min, imag_min, real_max, imag_max,
                scale_real, scale_imag, maxiter, 0);
```

In this version, the fill and compute cases are executed directly. The task is created in the recursive subdivision when the cutoff depth is reached:

Listing 15: Coarse task creation at the cutoff depth.

```

if (depth == CUTOFF) {
#pragma omp task firstprivate(M, hRows, hCols, y0, y1, x0, x1, CminR, CminI, CmaxR, CmaxI)
{
    mandel_tiled_rec(M, hRows, hCols, y0, x0,
                    CminR, CminI, CmaxR, CmaxI,
                    scale_real, scale_imag, maxiter, depth + 1);

    mandel_tiled_rec(M, hRows, hCols, y1, x0,
                    CminR, CminI, CmaxR, CmaxI,
                    scale_real, scale_imag, maxiter, depth + 1);

    mandel_tiled_rec(M, hRows, hCols, y0, x1,
                    CminR, CminI, CmaxR, CmaxI,
                    scale_real, scale_imag, maxiter, depth + 1);

    mandel_tiled_rec(M, hRows, hCols, y1, x1,
                    CminR, CminI, CmaxR, CmaxI,
                    scale_real, scale_imag, maxiter, depth + 1);
}
} else {
    mandel_tiled_rec(M, hRows, hCols, y0, x0,
                    CminR, CminI, CmaxR, CmaxI,
                    scale_real, scale_imag, maxiter, depth + 1);

    mandel_tiled_rec(M, hRows, hCols, y1, x0,
                    CminR, CminI, CmaxR, CmaxI,
                    scale_real, scale_imag, maxiter, depth + 1);

    mandel_tiled_rec(M, hRows, hCols, y0, x1,
                    CminR, CminI, CmaxR, CmaxI,
                    scale_real, scale_imag, maxiter, depth + 1);

    mandel_tiled_rec(M, hRows, hCols, y1, x1,
                    CminR, CminI, CmaxR, CmaxI,
                    scale_real, scale_imag, maxiter, depth + 1);
}

```

The same idea is used in the two-way split case, where only two recursive calls are grouped into the coarse task. No taskwait is required after creating these coarse tasks because the parent does not use a value produced by the children. The final barrier at the end of the parallel region waits for all pending tasks.

Histogram updates are still protected using atomic, and display operations are still protected with the named critical section `display_output`. Therefore, the change with respect to the basic leaf version is only the task granularity, not the output semantics.

3.2.3. Modelfactors Analysis

Overview of whole program execution metrics										
Number of threads	1	2	4	8	12	16	20	24	28	32
Elapsed time (sec)	1.81	1.16	0.77	0.77	0.77	0.77	0.77	0.77	0.77	0.77
Speedup	1.00	1.57	2.35	2.35	2.35	2.34	2.34	2.34	2.34	2.34
Efficiency	1.00	0.78	0.59	0.29	0.20	0.15	0.12	0.10	0.08	0.07

Table 1: Analysis done on Fri May 22 10:42:17 AM CEST 2026, par2314

Figure 18: Execution metrics for the improved leaf recursive version.

The improved leaf version creates only 10 explicit tasks, compared with 3013 tasks in the basic leaf version. The average task duration increases to approximately 170 ms, so each task contains much more useful work. This confirms that the cutoff successfully reduces task management overhead.

The maximum speedup improves from approximately 1.29 in the basic leaf version to approximately 2.35. However, the global efficiency is only 14.65% with 16 threads and 7.32% with 32 threads. This happens because 10 tasks are not enough to keep a large number of threads busy. The improved leaf version solves the overhead problem of the basic leaf strategy, but it introduces a lack of parallelism.

3.2.4. Paraver Analysis

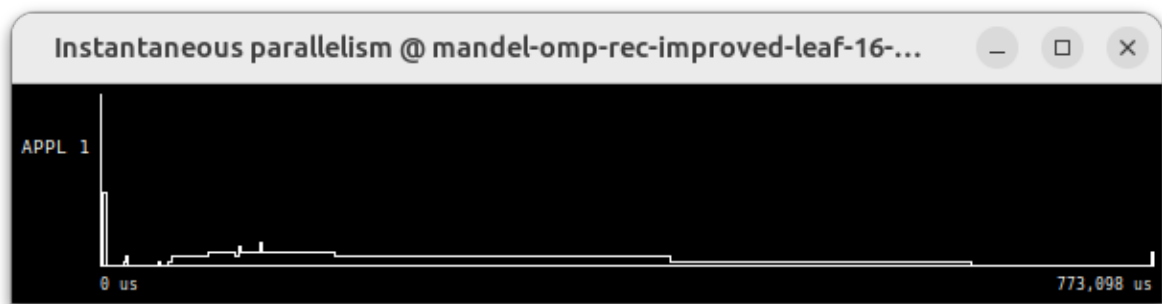


Figure 19: Instantaneous parallelism for the improved leaf recursive version with 16 threads.

The instantaneous parallelism view shows a clearer initial use of parallelism than the basic leaf version, but the amount of parallelism is still limited. The program cannot sustain high parallelism because the cutoff produces only a small number of coarse tasks.

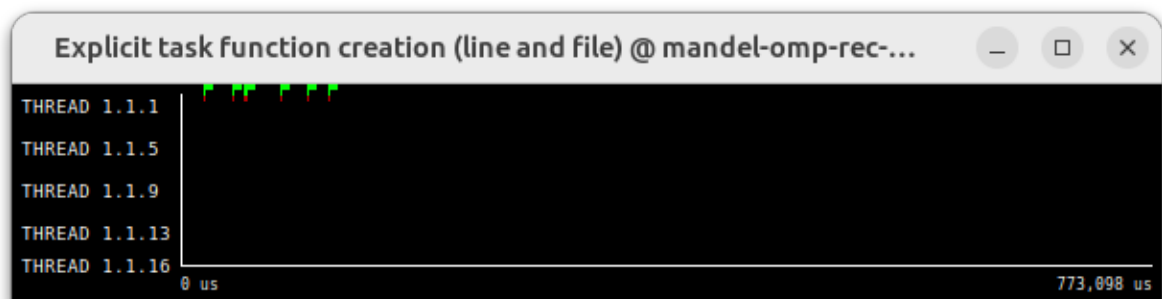


Figure 20: Task creation view for the improved leaf recursive version with 16 threads.

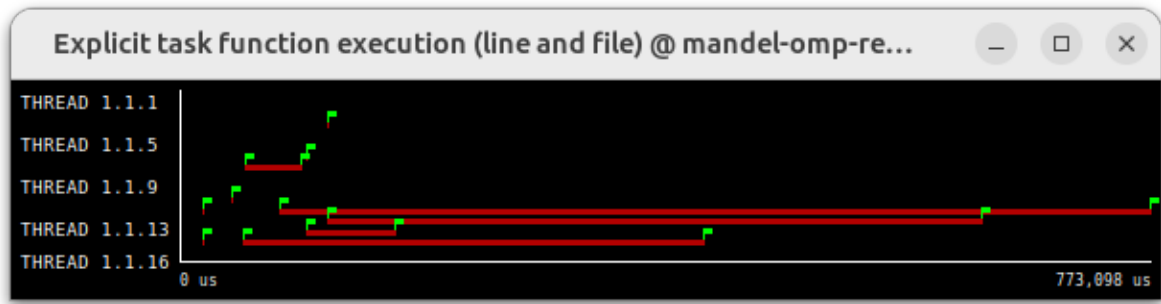


Figure 21: Task execution view for the improved leaf recursive version with 16 threads.

The task creation view shows very few task creation events, which is consistent with the 10 explicit tasks reported by Model factors. The task execution view shows long tasks, but many threads remain idle once there are no more independent tasks to schedule. Therefore, this strategy improves task granularity but does not provide enough task-level parallelism.

3.2.5. Strong Scaling

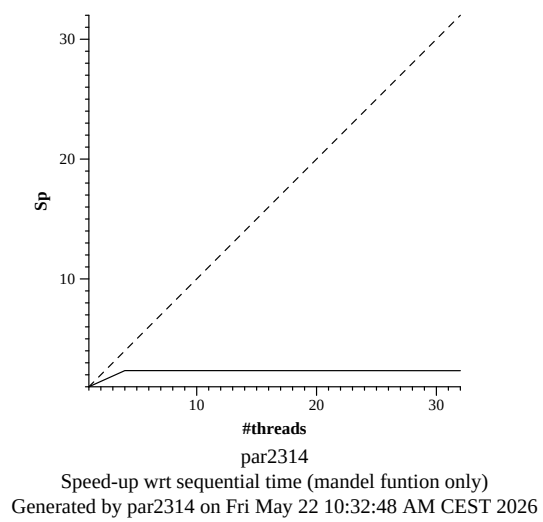


Figure 22: Strong scaling for the improved leaf recursive version.

Table 6: Strong scaling results for the improved leaf recursive version.

Threads	1	4	8	12	16	20	24	28	32
Time (s)	1.811823	0.771375	0.771556	0.771803	0.772021	0.772619	0.772236	0.772996	0.773311
Speedup	1.000	2.349	2.348	2.347	2.347	2.345	2.346	2.344	2.343

The improved leaf version is clearly better than the basic leaf version, reaching approximately 2.35 speedup. However, the speedup plateaus from 4 threads onward. This confirms the ModelFactors and Paraver observations: the cutoff reduces overhead, but the resulting number of tasks is too small for good scalability at high thread counts.

3.2.6. Comparison with the Basic Leaf Version

Compared with the basic leaf version, the improved leaf version changes the main bottleneck. The basic leaf version creates too many small tasks and exposes them too late. The improved version creates far fewer and larger tasks, reducing overhead and improving speedup. However, it overcorrects the problem: the task count is so small that the runtime cannot distribute enough work among all threads. Therefore, the improved leaf version is useful to demonstrate the effect of task granularity, but it is not the best recursive strategy.

3.3. Recursive Tree Strategy

3.3.1. Task Decomposition

The recursive tree strategy exposes parallelism during the recursive subdivision itself. Instead of waiting until the leaves, every recursive call that subdivides the current region creates tasks for its children. This makes parallel work available much earlier than in the leaf strategies.

In this version, the border equality check was also parallelized. The original shared equal variable was split into equal_vertical and equal_horizontal. This removes the race that would appear if both border-check tasks updated the same variable.

3.3.2. Source Code Implementation

The top-left pixel is computed first and used as the reference value:

Listing 16: Reference value used for the parallel border check.

```
int ref = pixel_dwell(COLS, ROWS, CminR, CminI, CmaxR, CmaxI,
                    x, y, scale_real, scale_imag, maxiter);
M[y][x] = ref;

int equal_vertical = non_first_call;
int equal_horizontal = non_first_call;
```

The vertical and horizontal border checks are executed as independent tasks:

Listing 17: Parallel border check in the tree version.

```
#pragma omp task firstprivate(M, NRows, NCols, x, y, ref, CminR, CminI, CmaxR, CmaxI,
{
    for (int py = y + 1; py < y + NRows - 1; py++) {
        M[py][x] = pixel_dwell(COLS, ROWS, CminR, CminI,
                                CmaxR, CmaxI, x, py,
                                scale_real, scale_imag, maxiter);
        M[py][x + NCols - 1] = pixel_dwell(COLS, ROWS, CminR, CminI,
                                            CmaxR, CmaxI, x + NCols - 1, py,
                                            scale_real, scale_imag, maxiter);

        equal_vertical = equal_vertical & (ref == M[py][x]);
        equal_vertical = equal_vertical & (ref == M[py][x + NCols - 1]);
```

```

    }
}

#pragma omp task firstprivate(M, NRows, NCols, x, y, ref, CminR, CminI, CmaxR, CmaxI, scale_real, scale_imag, maxiter)
{
    for (int px = x; px < x + NCols; px++) {
        M[y][px] = pixel_dwell(COLS, ROWS, CminR, CminI,
                               CmaxR, CmaxI, px, y,
                               scale_real, scale_imag, maxiter);
        M[y + NRows - 1][px] = pixel_dwell(COLS, ROWS, CminR, CminI,
                                              CmaxR, CmaxI, px, y + NRows - 1,
                                              scale_real, scale_imag, maxiter);

        equal_horizontal = equal_horizontal & (ref == M[y][px]);
        equal_horizontal = equal_horizontal & (ref == M[y + NRows - 1][px]);
    }
}

#pragma omp taskwait

equal = equal_vertical & equal_horizontal;

```

The taskwait is required here because the program immediately reads `equal_vertical` and `equal_horizontal` to decide whether the region can be filled or must be subdivided further. The vertical loop skips the corner pixels already handled by the horizontal task, avoiding concurrent writes to the same corner positions.

If the region is not uniform and is not yet at the base case, the recursive subdivision is taskified. In the four-way split case, each quadrant becomes a task:

Listing 18: Recursive task creation in the tree version.

```

#pragma omp task firstprivate(M, hRows, hCols, y0, x0, CminR, CminI, CmaxR, CmaxI,
                               mandel_tiled_rec(M, hRows, hCols, y0, x0,
                                                  CminR, CminI, CmaxR, CmaxI,
                                                  scale_real, scale_imag, maxiter));

#pragma omp task firstprivate(M, hRows, hCols, y1, x0, CminR, CminI, CmaxR, CmaxI,
                               mandel_tiled_rec(M, hRows, hCols, y1, x0,
                                                  CminR, CminI, CmaxR, CmaxI,
                                                  scale_real, scale_imag, maxiter));

#pragma omp task firstprivate(M, hRows, hCols, y0, x1, CminR, CminI, CmaxR, CmaxI,
                               mandel_tiled_rec(M, hRows, hCols, y0, x1,
                                                  CminR, CminI, CmaxR, CmaxI,
                                                  scale_real, scale_imag, maxiter));

```

```

mandel_tiled_rec(M, hRows, hCols, y0, x1,
                 CminR, CminI, CmaxR, CmaxI,
                 scale_real, scale_imag, maxiter);

#pragma omp task firstprivate(M, hRows, hCols, y1, x1, CminR, CminI, CmaxR,
mandel_tiled_rec(M, hRows, hCols, y1, x1,
                 CminR, CminI, CmaxR, CmaxI,
                 scale_real, scale_imag, maxiter);

```

No cutoff is used in this tree version. The purpose is to expose as much recursive parallelism as possible. No taskwait is needed after these recursive tasks because each task writes a disjoint subregion and the parent does not need to combine a result from its children. The barrier at the end of the parallel region waits for all outstanding tasks.

3.3.3. Modelfactors Analysis

Overview of whole program execution metrics										
Number of threads	1	2	4	8	12	16	20	24	28	32
Elapsed time (sec)	1.77	0.90	0.46	0.25	0.18	0.15	0.13	0.12	0.11	0.10
Speedup	1.00	1.97	3.83	7.11	9.65	11.93	13.63	15.29	16.54	17.87
Efficiency	1.00	0.99	0.96	0.89	0.80	0.75	0.68	0.64	0.59	0.56

Table 1: Analysis done on Fri May 22 10:44:13 AM CEST 2026, par2314

Figure 23: Execution metrics for the recursive tree version.

The tree version creates 12050 explicit tasks and 4017 taskwait/taskgroup synchronizations. This is a much larger task count than in the leaf versions, but the additional parallelism compensates for the overhead. The average task duration is approximately 118 microseconds at 16 threads, so tasks are fine-grained, but there are enough of them to keep the runtime busy.

The Modelfactors results show strong scalability. The speedup reaches 11.93 with 16 threads and 17.87 with 32 threads. Global efficiency remains 74.55% with 16 threads and 55.91% with 32 threads, much higher than both leaf strategies. Load balancing is also much better: at 16 threads it is 81.42%, while the improved leaf version reaches only 16.36%.

3.3.4. Paraver Analysis

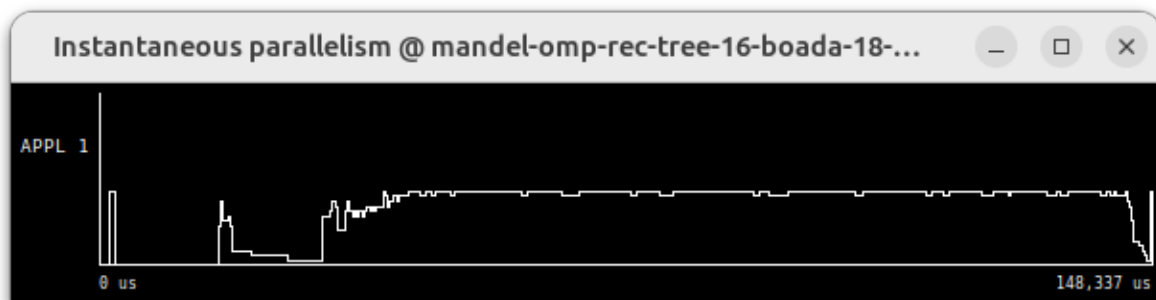


Figure 24: Instantaneous parallelism for the recursive tree version with 16 threads.

The instantaneous parallelism view shows an initial ramp-up while the recursive task tree is being generated. After that, the program sustains high parallelism for most of the execution. This is very different from the leaf strategies, where most of the execution had low thread utilization.

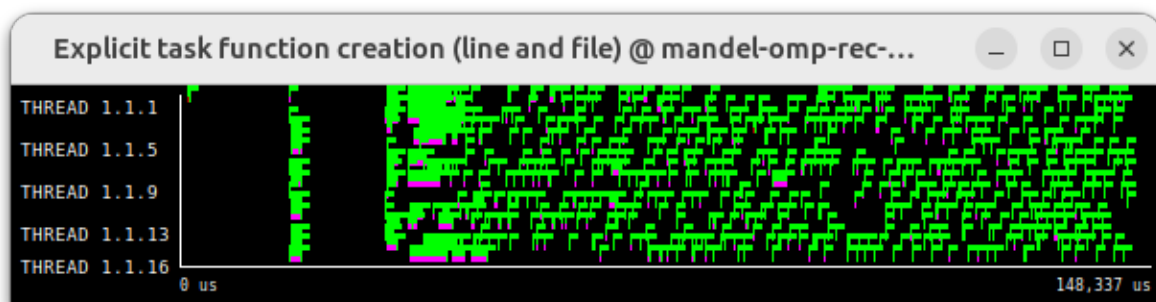


Figure 25: Task creation view for the recursive tree version with 16 threads.

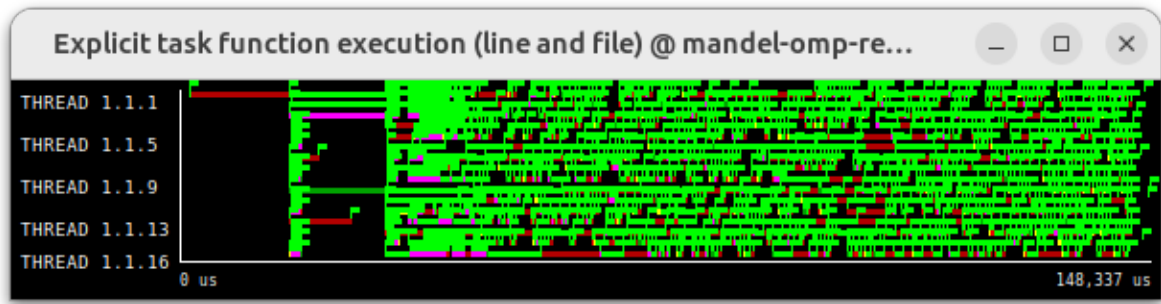


Figure 26: Task execution view for the recursive tree version with 16 threads.

The task creation and execution traces show dense activity across many threads. Tasks are not created only at the end of the recursion. Instead, recursive tasks create additional recursive tasks, so parallelism expands through the tree. This explains the much better load balance and the sustained parallelism observed in the instantaneous parallelism view.

3.3.5. Strong Scaling

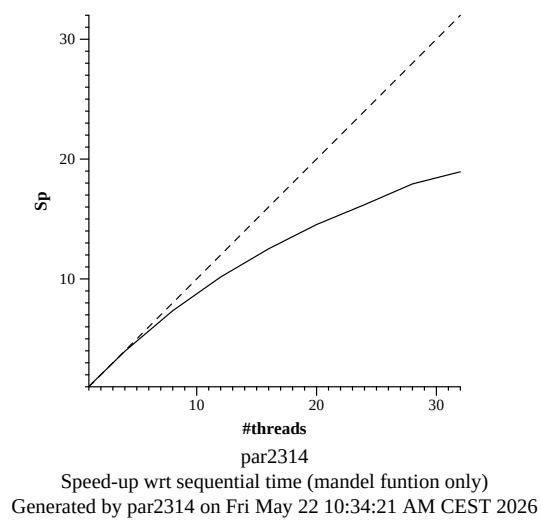


Figure 27: Strong scaling for the recursive tree version.

Table 7: Strong scaling results for the recursive tree version.

Threads	1	4	8	12	16	20	24	28	32
Time (s)	1.754725	0.457225	0.246529	0.178271	0.144781	0.124617	0.111875	0.101056	0.095641
Speedup	1.032	3.962	7.348	10.162	12.512	14.537	16.193	17.926	18.941

The tree version is the best recursive strategy. It reaches a speedup of 12.51 with 16 threads and 18.94 with 32 threads. Unlike the two leaf strategies, it continues improving as the number of threads increases. The reason is that it exposes parallelism throughout the recursion and provides enough tasks for the OpenMP runtime to distribute work effectively.

3.3.6. Comparison with Improved Leaf

The improved leaf version reduces task overhead by creating only 10 coarse tasks, but this limits scalability. The tree version takes the opposite approach: it creates many tasks, including tasks for recursive subdivision and border checks. This increases scheduling and synchronization overhead, but it also exposes enough parallelism to keep the machine busy. The strong scaling results show that, for this workload, the additional parallelism of the tree strategy is much more important than the extra overhead.

3.4. Comparison Between Recursive Strategies

Table 8: Execution time comparison between recursive strategies.

Threads	Leaf time (s)	Improved leaf time (s)	Tree time (s)
1	1.825030	1.811823	1.754725
4	1.405713	0.771375	0.457225
8	1.406957	0.771556	0.246529
12	1.407350	0.771803	0.178271
16	1.408503	0.772021	0.144781
20	1.408079	0.772619	0.124617
24	1.409933	0.772236	0.111875
28	1.408903	0.772996	0.101056
32	1.409785	0.773311	0.095641

Table 9: Speedup comparison between recursive strategies.

Threads	Leaf speedup	Improved leaf speedup	Tree speedup
1	0.993	1.000	1.032
4	1.289	2.349	3.962
8	1.288	2.348	7.348
12	1.287	2.347	10.162
16	1.286	2.347	12.512
20	1.287	2.345	14.537
24	1.285	2.346	16.193
28	1.286	2.344	17.926
32	1.285	2.343	18.941

The basic leaf version is the weakest recursive strategy. It creates a moderate number of small leaf tasks, but the recursive traversal itself remains sequential. Consequently, the useful parallelism is exposed too late and the speedup saturates at approximately 1.29.

The improved leaf version changes the task granularity. Instead of creating many small final-leaf tasks, it creates a much smaller number of coarse tasks at the cutoff depth. This reduces task overhead and improves speedup to approximately 2.35. However, the number of tasks becomes too small to keep many threads busy, so the scalability plateaus after 4 threads.

The recursive tree version is clearly the best recursive strategy. It exposes parallelism throughout the recursive decomposition, creates many more tasks, and obtains much better load balancing. Although this introduces more scheduling and synchronization overhead, the additional parallelism dominates, reaching a speedup of 18.94 with 32 threads.

4. Final Strategy Comparison

4.1. Comparison Table

4.2. Comparison between all Parallel Strategies

Table 10: Execution time comparison between all implemented strategies.

Version	1 thread	4 threads	8 threads	12 threads	16 threads	32 threads
Iterative original tile	3.462507	1.019928	0.732130	0.719861	0.720354	0.719224
Iterative fine grain	3.473498	0.871647	0.439175	0.294578	0.223028	0.115157
Recursive leaf	1.825030	1.405713	1.406957	1.407350	1.408503	1.409785
Recursive improved leaf	1.811823	0.771375	0.771556	0.771803	0.772021	0.773311
Recursive tree	1.754725	0.457225	0.246529	0.178271	0.144781	0.095641

Table 11: Speedup comparison between all implemented strategies.

Version	1 thread	4 threads	8 threads	12 threads	16 threads	32 threads
Iterative original tile	0.9997	3.3939	4.7280	4.8086	4.8053	4.8129
Iterative fine grain	0.9964	3.9708	7.8810	11.7495	15.5189	30.0559
Recursive leaf	0.993	1.289	1.288	1.287	1.286	1.285
Recursive improved leaf	1.000	2.349	2.348	2.347	2.347	2.343
Recursive tree	1.032	3.962	7.348	10.162	12.512	18.941

Table 12: Qualitative summary of the implemented strategies.

Version	Task granularity	Main limitation	Conclusion
Iterative original tile	Coarse tile tasks	Only 64 tasks are created, and their cost is highly irregular.	Correct but limited by load imbalance. Speedup saturates after 8–12 threads.
Iterative fine grain	Border-check tasks and row-level tasks	Higher task-management overhead than original tile.	Best iterative strategy. It exposes enough parallelism and achieves almost linear scaling up to high thread counts.
Recursive leaf	Tasks only at final recursive leaves	Parallelism is exposed too late and tasks are too fine-grained.	Correct but not scalable. Speedup stays around 1.29.
Recursive improved leaf	Coarse tasks at cut-off depth	Too few tasks are created to use many threads.	Improves over basic leaf by reducing task overhead, but speedup plateaus around 2.35.
Recursive tree	Tasks throughout the recursion tree	More task creation and synchronization overhead.	Best overall execution time. It exposes recursive parallelism early and reaches the fastest time at 32 threads.

4.3. Conclusions

This laboratory compared five OpenMP task decomposition strategies for the Mandelbrot computation: two iterative versions and three recursive versions. The results show that task granularity and the moment when parallelism is exposed are the most important factors for scalability.

The original iterative tile version creates only one task per tile. This produces a small number of coarse tasks, so the OpenMP overhead is low, but the workload is very unbalanced. Some tiles require much more computation than others, and once the short tasks finish, several threads remain idle. For this reason, the speedup saturates around 4.8.

The fine-grain iterative version solves this limitation by creating more tasks inside each tile. It parallelizes the border checks and the row-level fill or compute work. This increases the number of tasks but greatly improves load balancing. As a result, it is the best iterative strategy, reaching a speedup of 30.06 with 32 threads and an execution time of 0.115157 seconds.

The basic recursive leaf version performs poorly. Although it creates several tasks, they are created only at the leaves of the recursion. This delays the exposure of parallelism and creates many small tasks with limited useful work. Its speedup stays around 1.29 independently of the number of threads.

The improved leaf version introduces a cutoff to create coarser recursive tasks. This reduces task creation overhead and improves the speedup to approximately 2.35. However, the improvement is limited because the cutoff creates too few tasks, so the program cannot use many threads effectively.

The recursive tree version gives the best overall execution time. By creating tasks during the recursive subdivision, it exposes parallelism much earlier than the leaf strategies. It also parallelizes the border equality check by splitting the original shared equal variable into independent horizontal and vertical checks. This version creates many more tasks and introduces more synchronization, but the additional parallelism and better load balancing compensate for that overhead. It reaches 0.095641 seconds with 32 threads, which is the fastest execution time among all implementations.

Therefore, the best implementation in terms of absolute performance is the recursive tree strategy. The best iterative implementation is the fine-grain version. If simplicity is prioritized, the fine-grain iterative version is attractive because it gives strong scalability with a simpler structure than the recursive tree. However, if the objective is the minimum execution time, the recursive tree version is the best strategy in our experiments.